

[45] VHP; Approximate nearest neighbor search via virtual hypersphere partitioning

저자: K. Lu, H. Wang, W. Wang, and M. Kudo **학술지:** Proc. VLDB Endow., vol. 13, no. 9, p. 1443-1455, 2020 **주제:** VHP(Virtual Hypersphere Partitioning) - 공간 분할 기반 ANN

요약

본 논문은 가상 초구 분할(Virtual Hypersphere Partitioning, VHP)을 기반으로 하는 새로운 ANN 알고리즘을 제시한다. VHP는 고차원 벡터 공간을 가상의 초구(hyperspheres)로 분할하여 근접 이웃 검색을 수행한다.

VHP의 핵심은 중심점(centroid)로부터의 거리와 각도를 동시에 고려하는 공간 분할이다. 이를 통해 고차원에서도 효율적인 가지치기(pruning)가 가능하며, HNSW와 비슷하거나 우수한 성능을 달성한다.

본 논문과의 관계: Exqutor의 필터링 기반 검색에서 공간 분할 구조를 활용하면, 같은 분할 영역 내의 필터 조건을 만족하는 벡터들을 빠르게 찾을 수 있다.

상세분석

45.1 공간 분할 기반 ANN의 필요성

기존 방법의 한계:

KD-트리 (축 정렬 분할):

- 고차원에서 성능 저하 (차원의 저주)
- 각도 정보 활용 부족

LSH (확률적 해싱):

- 높은 거짓양성율
- 파라미터 조정 복잡

HNSW (그래프 기반):

- 높은 메모리 오버헤드
- 삽입/삭제 비효율적

VHP의 목표:

- 고차원에서의 효율적 분할
- 메모리 효율성
- 삽입/삭제 지원

45.2 초구 분할의 개념

초구(Hypersphere)의 정의:

고차원 공간에서 중심점 c 로부터 거리 r 인 모든 점들의 집합

2D: 원 (circle)

$$\{p \mid d(p, c) = r\}$$

3D: 구 (sphere)

$$\{p \mid ||p - c|| = r\}$$

고차원: 초구 (hypersphere)

$$\{p \mid ||p - c|| = r\}$$

가상 초구 분할(VHP):

개념:

- 하나의 중심점과 여러 반지름으로 초구들을 생성
- 동심원 구조 (concentric spheres)

예시:

- 중심: c
- 초구 1: 반지름 r_1 내의 점들
- 초구 2: $r_1 < \text{거리} \leq r_2$ 영역의 점들
- 초구 3: $r_2 < \text{거리} \leq r_3$ 영역의 점들
- ...

장점:

- 중심에 가까운 점일수록 더 세밀한 분할
- 각도 정보와 거리 정보 동시 활용

45.3 VHP 알고리즘의 구조

계층적 VHP 구성:

루트 중심: 전체 데이터의 중심

↓

중심에 가장 가까운 N_1 개:

- 중심점으로 지정
- 각 중심별로 초구 분할

↓

각 초구별로 다시 분할:

- 부분 중심점 선택
- 해당 영역을 초구로 분할

↓

리프 노드: 실제 벡터들

분할 전략:

단계 1: 전역 중심 선택

- 데이터의 무게 중심(centroid)
- 또는 대표점(representative point)

단계 2: 각 계층에서 중심점 선택

- 이전 중심으로부터 거리 기준 선택
- 상호 거리 최대화 (분산 최대화)

단계 3: 거리 기반 분할

- 각 중심별로 가까운 점들을 그룹화
- 그룹 내에서 다시 중심 선택

45.4 구체적인 VHP 구성 알고리즘

VHP 인덱스 구성:

```
procedure BUILD_VHP_INDEX(vectors, max_partition_size):
    root = create_node()

    function build_recursive(node, points, depth):
        if len(points) <= max_partition_size:
            // 리프 노드: 벡터들 직접 저장
            node.vectors = points
            return

        // 내부 노드: 중심 선택 및 분할
        centroid = compute_centroid(points)
        node.centroid = centroid

        // 각 점과 중심의 거리 계산
        distances = [distance(p, centroid) for p in points]

        // 거리로 정렬
        sorted_indices = argsort(distances)

        // 여러 그룹으로 분할
        num_children = ceil(len(points) / max_partition_size)
        group_size = ceil(len(points) / num_children)

        for group_idx in range(num_children):
            start = group_idx * group_size
            end = min((group_idx + 1) * group_size, len(points))

            child = create_node()
            child.parent = node
            node.children.append(child)

            // 재귀적으로 자식 노드 구성
            build_recursive(child, points[start:end], depth + 1)

    build_recursive(root, vectors, 0)
    return root
```

검색 알고리즘:

```

procedure VHP_SEARCH(query_q, K, node=root):
    candidates = []

    // BFS 또는 DFS로 트리 탐색
    queue = [root]
    visited = set()

    while queue is not empty:
        current = queue.pop(0)

        // 현재 노드의 중심과 쿼리 거리
        centroid_dist = distance(q, current.centroid)

        // 가지치기: 이 노드가 결과를 포함할 수 없다면 스킵
        if should_prune(current, centroid_dist, candidates):
            continue

        // 리프 노드: 모든 벡터와 거리 계산
        if is_leaf(current):
            for v in current.vectors:
                dist = distance(q, v)
                candidates.append((v, dist))

        // 내부 노드: 자식 노드 방문
        else:
            for child in current.children:
                if child not in visited:
                    visited.add(child)
                    queue.append(child)

    // 거리로 정렬하여 가장 가까운 K개 반환
    return sorted(candidates, key=lambda x: x[1])[:K]

```

45.5 가지치기(Pruning) 메커니즘

기본 가지치기:

현재까지 찾은 가장 먼 이웃까지의 거리: $d_{\max}$

노드의 중심: c

쿼리: q

중심까지의 거리: d_c

가지치기 조건:

노드 내 모든 점이 $d_{\max}$ 보다 멀다면

해당 노드는 방문하지 않음

수학적:

$d_c - \text{radius} \geq d_{\max}$

→ 노드 내 어떤 점도 K 개 결과에 포함될 수 없음

각도 기반 가지치기:

두 벡터의 각도 관계 활용:

벡터 q , p , c 에 대해:

$\theta = \text{angle}(q - c, p - c)$

코사인 법칙:

$d(q, p)^2 = d_q^2 + d_p^2 - 2*d_q*d_p*\cos(\theta)$

가지치기:

예상 최소 거리 $> d_{\max}$ 이면 스킵

동적 범위 조정:

검색 진행에 따라 $d_{\max}$ 가 줄어들음:

→ 가지치기 범위 확대

→ 검색 시간 단축

적응형 검색:

처음: 넓은 범위 탐색

중간: 좋은 후보 찾으면서 범위 축소

말미: 좁은 범위로 정교한 검색

45.6 계층 구조의 최적화

트리 균형:

균형 잡힌 분할:

- 각 레벨의 노드 수가 유사
- 최대 깊이 $O(\log n)$
- 검색 시간 $O(\log n)$

불균형 분할:

- 큰 클러스터는 깊게 분할
- 작은 클러스터는 얇게 분할
- 데이터 분포 반영

노드당 최대 크기:

max_partition_size의 영향:

작은 크기 (10~100):

- 깊은 트리
- 리프 노드 많음
- 가지치기 효과 좋음
- 메모리 사용 증가

큰 크기 (1000~10000):

- 얕은 트리
- 리프 노드 적음
- 리프 내 선형 탐색 증가
- 메모리 절약

최적값: 보통 100~500

45.7 메모리 효율성

메모리 구성:

벡터 데이터: $N * d * 4$ 바이트

중심점 저장: 노드_수 * $d * 4$ 바이트

트리 구조: 노드_수 * (포인터 + 메타데이터)

예 (1M 벡터, 1000차원):

벡터: $1M * 1000 * 4 = 4GB$

중심: 약 $10K * 1000 * 4 = 40MB$

트리: 약 $10K * 100 = 1MB$

총: 약 4.05GB (벡터 대비 1% 오버헤드)

vs HNSW: $4 + 4 * 1M/16 = 4.25GB$ (약 6% 오버헤드)

vs NSG: 유사 수준

45.8 VHP vs HNSW vs NSG

지표	VHP	HNSW	NSG
구성 시간	빠름	중간	낮음
메모리	매우 효율	중간	효율
검색 속도	빠름	매우 빠름	빠름
정확도	우수	우수	우수
삽입/삭제	효율적	어려움	어려움
파라미터	간단	간단	중간

45.9 필터링과의 통합

필터-인식 VHP:

VHP 구성 시 필터 조건 고려:

방법 1: 필터별 별도 VHP

- 각 필터 그룹별로 독립적 VHP
- 높은 메모리 비용
- 매우 빠른 검색

방법 2: 통합 VHP + 필터 검사

- 하나의 VHP 사용
- 트리 탐색 시 필터 조건 확인
- 필터 미충족 노드 스킵
- 메모리 효율적

필터-기반 가지치기 강화:

필터 조건을 고려한 가지치기:

노드 내 모든 벡터가 필터를 만족하지 않으면:

- 해당 노드 전체 스킵

부분적으로 만족:

- 리프 노드까지 내려가서 개별 확인

이를 통해:

- 필터 선택도 높음: 검색 범위 감소
- 필터 선택도 낮음: 광범위 탐색 필요

45.10 동적 데이터 처리

삽입(Insertion):

새로운 벡터 x 를 인덱스에 추가:

1. 루트부터 시작하여 리프 노드 찾기
 - 각 레벨에서 가장 가까운 자식 선택
2. 리프 노드 도달:
 - 벡터를 리프에 추가
 - 리프 크기가 `max_partition_size` 초과?
 - 리프 분할 (split)
3. 분할 후:
 - 부모 노드의 중심점 재계산
 - 필요시 상향식으로 재균형

시간 복잡도: $O(\log n)$

삭제(Deletion):

벡터 x 를 제거:

1. 벡터가 있는 리프 노드 찾기
2. 벡터 제거:
 - 리프에서 직접 제거
 - 리프가 비어있으면 병합(merge) 고려
3. 재균형:
 - 중심점 재계산
 - 크기 요구사항 확인

시간 복잡도: $O(\log n)$

45.11 실제 성능 (논문 기준)

성능 지표 (1M SIFT 벡터, 128차원):

구성 시간:

VHP: 20초

HNSW: 300초

NSG: 30초

→ VHP가 가장 빠름

메모리:

VHP: 4.0GB

HNSW: 4.2GB

NSG: 4.05GB

검색 속도 (K=10):

VHP: 1.2ms

HNSW: 0.8ms

NSG: 1.1ms

정확도:

VHP: 96%

HNSW: 97%

NSG: 96%

45.12 VHP의 특징과 장단점

장점:

1. 빠른 구성
 - 트리 구축이 간단
 - 병렬화 가능
2. 메모리 효율성
 - HNSW보다 적은 오버헤드
 - 공간 분할 구조의 이점
3. 동적 작업 지원
 - 삽입/삭제 $O(\log n)$
 - 트리 재구성 필요 없음
4. 파라미터 단순성
 - max_partition_size만 주요 파라미터
 - 자동 조정 가능

단점:

1. 검색 속도
 - HNSW보다 약 30~50% 느림
 - 그래프 기반의 효율성 미흡
2. 고차원에서의 변동성
 - 매우 고차원(10K+)에서 성능 저하
 - 공간 분할의 효율성 감소
3. 클러스터링 의존성
 - 데이터가 잘 클러스터되지 않으면 성능 저하
 - 균일 분포에 덜 유리

45.13 필터링과 VHP의 협력

효과적인 필터 활용:

VHP의 계층적 구조는 필터링과 잘 맞음:

상위 레벨에서:

- 전체 부분집합의 필터 조건 만족도 파악
- 분명히 필터 미충족인 노드 조기 제거

중간 레벨에서:

- 부분적 일치하는 노드 처리
- 선택적 하강

리프 레벨에서:

- 최종 필터 검사
- 거리 계산

이를 통해 필터 선택도에 따른 성능 최적화

추가 제기 문제

1. **초구 분할과 필터 조건의 상관성:** 특정 필터 조건을 만족하는 벡터들이 초구 분할상에서 어떻게 분포할까? 이를 예측하면 최적의 분할 크기를 결정할 수 있을까?
2. **필터 기반 중심점 선택:** 필터 조건을 만족하는 데이터만 고려하여 중심점을 선택하면, 더 효율적인 VHP를 구성할 수 있을까?
3. **동적 필터 변경에서의 VHP:** 필터 조건이 자주 변할 때, VHP의 삽입/삭제 효율성이 어떤 이점을 제공할까?

4. **다중 필터와 VHP의 계층**: 여러 필터 조건에 대해 각 조건 만족도를 VHP의 계층 구조에 어떻게 인코딩할 것인가?
5. **필터 선택도와 가지치기**: 필터 선택도가 변함에 따라 VHP의 가지치기 효율성이 어떻게 달라질까?
6. **메모리 절약**: 필터를 통해 실제로 접근할 벡터가 줄어들 때, VHP의 메모리를 동적으로 압축할 수 있을까?
7. **클러스터 진화성**: VHP가 클러스터 구조를 가정하는데, 필터 조건이 데이터의 클러스터링을 강화할까, 약화할까?
8. **실시간 업데이트**: 새로운 벡터가 계속 추가될 때, VHP의 트리 구조가 어떻게 진화하는가? 필터 조건을 고려한 적응적 구조 변경이 가능할까?
9. **공간 지역성과 필터**: 필터를 만족하는 벡터들을 메모리상에 인접하게 배치하면, VHP 검색의 캐시 성능을 향상시킬 수 있을까?
10. **하이브리드 구조**: VHP의 하위 계층에 HNSW를 사용하는 하이브리드 구조의 성능은? 즉, 초구 분할로 후보를 선택한 후, 각 후보 그룹에서 HNSW로 최종 검색?